touying-ufac-unofficial

Slides for the Federal University of Acre (UFAC): lectures, courses, coursework and theses.

v0.3.0 November 04, 2026

https://github.com/lucaslrodri/touying-ufac-unofficial

Lucas Lima Rodrigues

A Touying theme for the slides of the Federal University of Acre: lectures, courses, the work of students, final papers and theses. Slides are written with headings and a few Markdown-like shorthands for boxes, quotations, arrow lists and emphases. This manual is set with the rules of the theme itself.

Contents

1 Getting started 2
1.1 Install the font 2
1.2 A first deck 2
1.3 Dependencies 2
2 The information of the deck 3
2.1 The fields 3
2.2 The kind of work and the aliases 3
3 The cover and the closing slide 6
3.1 The cover 6
3.2 The closing slide 7
4 Building the deck 9
4.1 Sections and their figure 9
4.2 The outline 9
4.3 Slides and continuations 10
5 Exercises and examples 13
6 Writing on a slide 16
6.1 Emphasis 16
6.2 Code chips 16
6.3 Colors by context 17
6.4 Lists and terms 17
6.5 Columns 17
6.6 Escapes 17
7 Boxes 19
7.1 Emphasis boxes 19
7.2 Equation boxes 19
7.3 Callouts 19
7.4 Tags and quotations 20
8 Tables, figures, math and code 21
8.1 Tables 21
8.2 Figures and math 21
8.3 Code 21
9 Customization 23
9.1 Language 23
9.2 Colors 23
9.3 Logos 24
9.4 The footer and the badges 24
9.5 The information anywhere: `#info` 25
10 Integrations 26
10.1 Touying 26
10.2 Code blocks with codly 27
10.3 Icons 28
11 Moving from 0.2.0 30
12 Reference 31
12.1 The theme and the slides 31
12.2 The information of the deck 38
12.3 Boxes and emphases 39
12.4 Lists, columns and the outline 45
12.5 Code 47
12.6 Icons 47
12.7 Colors and logo 48
13 Index of definitions 50

1 Getting started

1.1 Install the font

The theme uses the font **New Computer Modern Sans**, which is not bundled with Typst. Without it, Typst falls back to another font and the slides do not look like the ones in this manual. It belongs to the New Computer Modern family: download the archive from CTAN and take the `NewCMSans10-*.otf` files of its `otf` folder. Install them on the system, or keep them in a folder and compile with `typst compile --font-path <folder> main.typ`. In the Typst web app, upload the `.otf` files to the project.

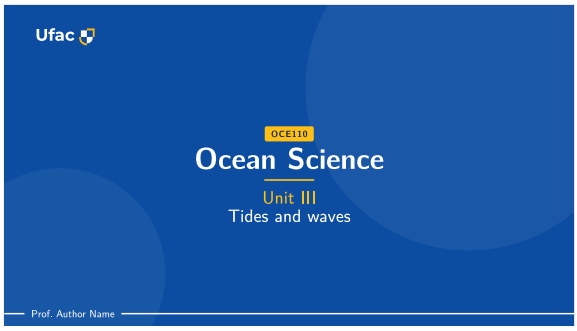
1.2 A first deck

`typst init @preview/touying-ufac-unofficial:0.3.0` creates a deck from the template of the package. A deck imports Touying and the theme, applies `ufac-theme` with the information of the deck, and starts with the cover:

```
#import "@preview/touying:0.7.4": *
#import "@preview/touying-ufac-unofficial:0.3.0": *
```

```
#show: ufac-theme.with(
  aspect-ratio: "16-9",
  lang: "en",
  config-info(
    subject: [Ocean Science],
    subject-code: [OCE110],
    unit-title: [Tides and waves],
    unit: [Unit III],
    author: [Prof. Author Name],
    counter-prefix: [1.],
  ),
)

#title-slide()
```



From here on, every example shows the source on the left and the result on the right: a real slide when the feature belongs to the slide (headers, pills, continuations), otherwise the content alone, laid out by the same rules as in a deck. The repository has complete decks to start from: a lecture, an extension course, a final paper and the work of students for a course (`example/`).

1.3 Dependencies

The theme needs **Typst 0.14.0 or later**. Typst fetches the packages by itself:

Package	Version	Role
`touying`	`0.7.4`	the slide engine; the deck imports it too, with the same version
`iconify`	`0.5.3`	the icons of a collection the deck gives to `provide-icons`; the theme's own icons ship with the theme
`codly`	`2.0.0`	optional: the code blocks through codly (requires Typst 0.15)

touying 0.8 requires Typst 0.15; this version of the theme stays on touying 0.7.4, which also works with Typst 0.14. codly is imported by the deck that wants it (Section 10.2).

2 The information of the deck

2.1 The fields

The cover, the footer and the metadata of the PDF come from Touying's `config-info`, given to `ufac-theme`. The theme reads these fields:

Field	Default	What it is and where it appears
`title`	`[]`	the name of the course or of the work: large on the cover, and in the footer
`short-title`	`none`	its code or a shorter name: the tag of the cover, and before the title in the footer
`subtitle`	`[]`	the topic of the lecture: on the cover
`short-subtitle`	`none`	the lecture or the unit ("Lecture 1", "Unit II"): on the cover, in yellow
`author`	`none`	one, an array or free content: on the cover and on the right of the footer
`date`	`none`	`auto` (today), a `datetime` (written in full in the language of the deck) or a text
`institution`	`auto`	one or an array; `auto` is the Universidade Federal do Acre; shown with `#info` (Section 9.5)
`logo`	`auto`	the logos of the footer and of the cover (Section 9.3)
`contact`	`none`	one or an array (an e-mail, a page); shown with `#info` (Section 9.5)
`counter-prefix`	`[1.]`	prefix of the numbers of exercises and examples: "Exercise 1.N"; `none` gives "Exercise N", the default of the kinds without units (Section 2.2)

An author written as free content keeps its lines (broken with `\`), and a line only in emphasis is a label over a group: the cover shows it as written, and the footer and the metadata get the names of the first group, those who present.

```
config-info(author: [
  _Students_ \ First Author \ Second Author \
  _Advisor_ \ Prof. Dr. Advisor Name
])
```

2.2 The kind of work and the aliases

In a course, the fields have names of their own, which are aliases of the fields above; giving one field under two names (`subject` and `title`) is an error. A date given as `semester` or `year` is short, and also goes to the footer, after the title.

Alias	Field
`subject`	`title`
`subject-code`	`short-title`
`unit`	`short-subtitle`
`unit-title`	`subtitle`
`course` (the program, in a final paper)	`short-subtitle`
`semester`, `year`	`date`

`kind` tells what the presentation is, and brings the recommended cover and numbering; anything the deck writes wins over it. The prefix of the exercises and examples ("Exercise 1.1") stands for the unit of a

course, so the kinds without units number them without it (“Exercise 1”). Without a `kind`, the cover has the style `"line"` and the numbers have the prefix `[1.]`.

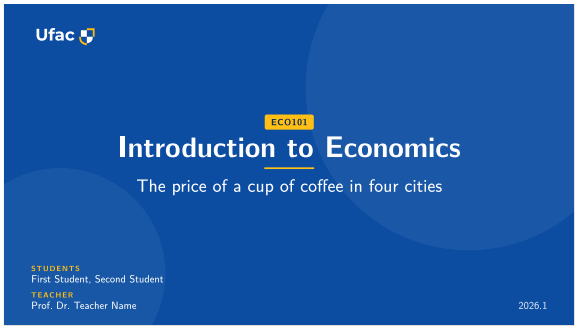
`kind`	Recommends
`"lecture"`	the style `"line"`: the authors and the date on one line at the bottom
`"course"`	the style `"top"` and the label “Instructors” (“Ministrantes”) over the authors
`"talk"`	the style `"line"`; no units: exercises without a prefix
`"coursework"`	the style `"stack"`: the authors in groups at the bottom left; no units: exercises without a prefix
`"tcc"`, `"dissertation"`, `"thesis"`	the style `"top"` and the tag of the work (“Undergraduate Thesis”, …); no units: exercises without a prefix

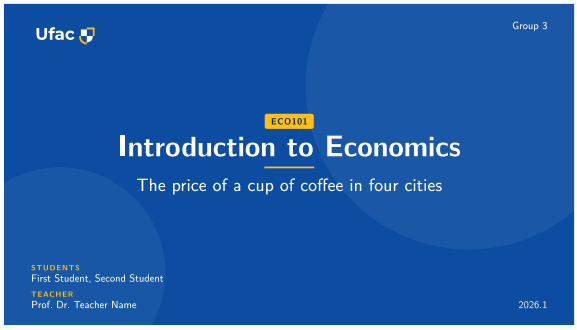
The work of students for a course keeps the names of the course and takes `kind: "coursework"`: the students and the teacher stack at the bottom left, each group under its label, and the top right is free, for the name of the group.

```
#show: ufac-theme.with(
  lang: "en",
  config-info(
    kind: "coursework",
    subject: [Introduction to Economics],
    subject-code: [ECO101],
    unit-title: [The price of a cup of
      coffee in four cities],
    author: [
      _Students_ \
      First Student, Second Student \
      _Teacher_ \
      Prof. Dr. Teacher Name
    ],
    semester: [2026.1],
  ),
)

#title-slide()

#title-slide(header-right: [Group 3])
```



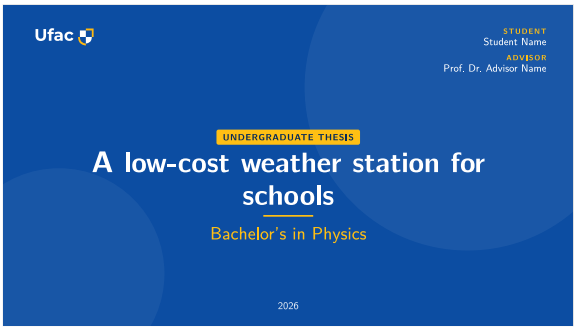


A final paper takes `kind: "tcc"`: the tag of the work over the title, the program as `course`, and the student and the advisors at the top right. `"dissertation"` and `"thesis"` do the same with their own tag.

Reference: `ufac-theme()` p. 31

```typst
#show: ufac-theme.with(
  lang: "en",
  config-info(
    kind: "tcc",
    title: [A low-cost weather station
      for schools],
    course: [Bachelor's in Physics],
    author: [
      _Student_ \ Student Name \
      _Advisor_ \ Prof. Dr. Advisor Name
    ],
    year: 2026,
  ),
)

#title-slide()
```



3 The cover and the closing slide

3.1 The cover

`#title-slide()` draws the cover in three bands: the logos at the top left, the titles in the center, and the authors and the date. The `style` decides where the authors and the date go: `"line"` (the default) puts them on one line at the bottom, with the white line of the footer passing between them; `"stack"` stacks the authors at the bottom left; `"top"` puts them at the top right and the date in the center of the bottom. Each of the six positions can be changed on its own, with content, `none` or a function of `self`, the dictionary Touying keeps of the deck (`self => self.info.title`).

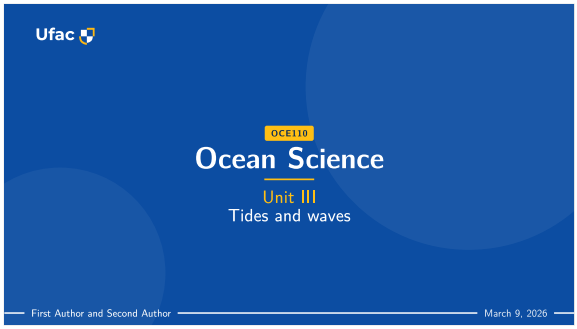
```
#show: ufac-theme.with(
  lang: "en",
  config-info(
    subject: [Ocean Science],
    subject-code: [OCE110],
    unit-title: [Tides and waves],
    unit: [Unit III],
    author: ([First Author],
      [Second Author]),
    date: datetime(
      year: 2026, month: 3, day: 9),
  ),
)

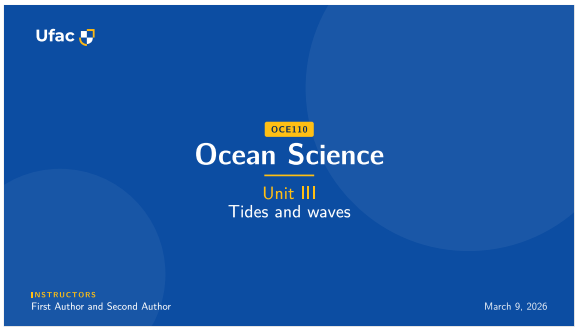
#title-slide()   // style: "line"

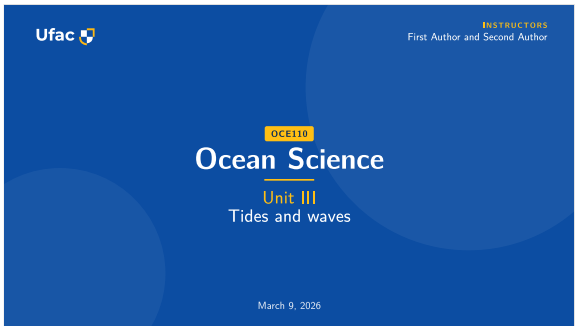
#title-slide(style: "stack",
  author-label: [Instructors])

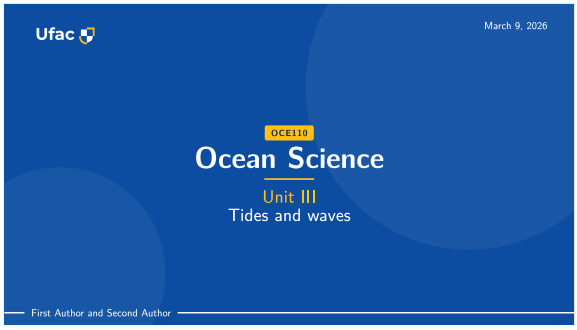
#title-slide(style: "top",
  author-label: [Instructors])

#title-slide(footer-right: none,
  header-right: info("date"))
```





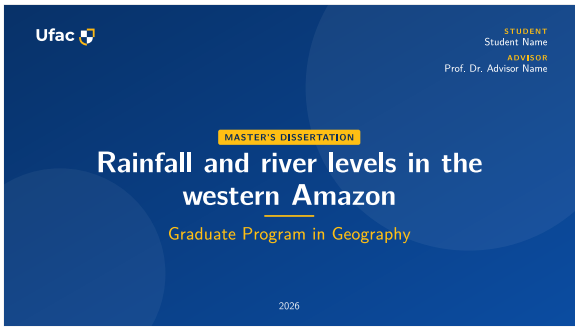




The background of the cover and of the closing slide is an option of the theme: `"circles"` (the default), `"gradient"` or `"plain"`; here, on the cover of a master's dissertation.

```
#show: ufac-theme.with(
  lang: "en",
  background: "gradient",
  config-info(
    kind: "dissertation",
    title: [Rainfall and river levels in
      the western Amazon],
    course: [Graduate Program in
      Geography],
    author: [
      _Student_ \ Student Name \
      _Advisor_ \ Prof. Dr. Advisor Name
    ],
    year: 2026,
  ),
)

#title-slide()
```



Content given to the cover goes below the titles, in white: a message, where to find the slides, a contact.

Reference: `title-slide()` p. 33

```
#title-slide[
  Slides and exercises:
  #link("https://example.com/ocean")[
    `example.com/ocean`] \
  Questions: *`author@example.com`*
]
```



3.2 The closing slide

`#ending-slide` closes the deck on the blue page of the cover: a large title with the yellow rule, a subtitle in yellow and a free body, with the footer of the slides in white. Without a title it says "Thank you!" in the language of the deck. The contact of the deck goes in the body, with `#info("contact")` (Section 9.5).

Reference: `ending-slide()` p. 36

```
#ending-slide()

#ending-slide[Questions?][Next: tides][
  #info("contact")
]

#ending-slide(title: none)[
  *Break*: 15 minutes
]
```



Questions?

Next: tides

author@example.com

Ufac · OCE110 • OCEAN SCIENCE · PROF. AUTHOR NAME · 2

Break: 15 minutes

Ufac · OCE110 • OCEAN SCIENCE · PROF. AUTHOR NAME · 3

4 Building the deck

Headings build the deck: no function call is needed for the ordinary slides.

Markup	Effect
`= Title`	section slide: the title under the yellow tag “PART N”; what follows the heading goes to the right
`== Title`	new slide with a blue header, fitted to the width
`==`	a slide without a title, with the footer
`== Title <exercise>`, `<example>`	exercise or example: a box filling the slide, numbered (Section 5)
`=== Subtitle`	a pill; no new slide; repeated at the top of every continuation page
`==== Subtitle`	the same pill, not repeated
`---`, `#pagebreak()`	next page of the same slide (same header; in an exercise or example, the same box and number)

4.1 Sections and their figure

Every `=` heading opens a section slide: “PART N” in the yellow tag, the title in blue and the yellow rule. The content right after the `=`, typically an image, sits on the right, one page margin away from the edge, in a frame of 12em × 66% of the height of the content area. An image without a size fills the frame, scaled up or down with its proportions. An image with a size keeps it, and shrinks whole when it is larger than the frame, so a small image or an `#icon` keeps its size; a size in percent (`width: 100%`, `height: 50%`) is a share of the frame. The image is neither cropped nor stretched, also inside a figure, an `align` or a link.

Reference: `new-section-slide()` p. 35

```
= The coast
#image("figure.svg")
```



4.2 The outline

`#outline()` lists the parts of the deck, each line a link to the slide where the part starts. It usually takes a slide of its own, right after the cover.

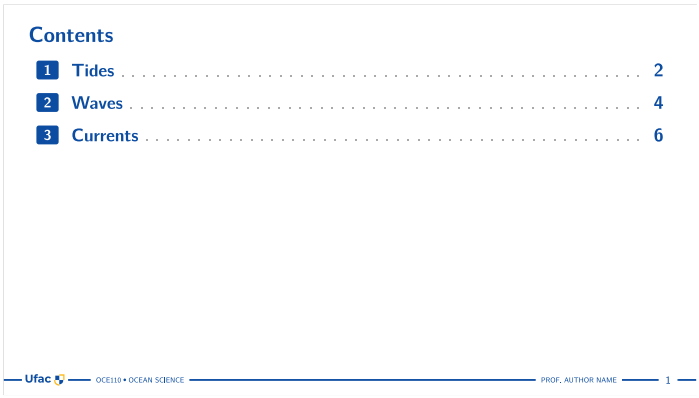
Reference: `outline()` p. 47

```
== Contents
#outline()

= Tides
== The pull of the Moon
Two high tides a day.

= Waves
== Wind
Longer waves travel faster.

= Currents
== The Gulf Stream
A river inside the ocean.
```



4.3 Slides and continuations

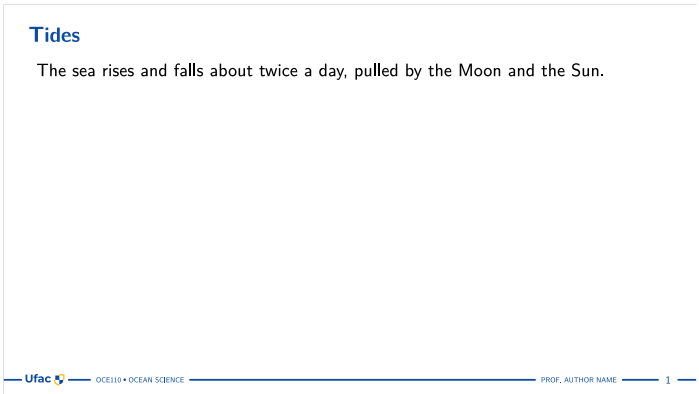
A `===` subtitle opens a pill without a new slide. A `---` alone on a line goes on to the next page of the same slide, with the same header (the number in the footer counts the pages); a long content goes on by itself.

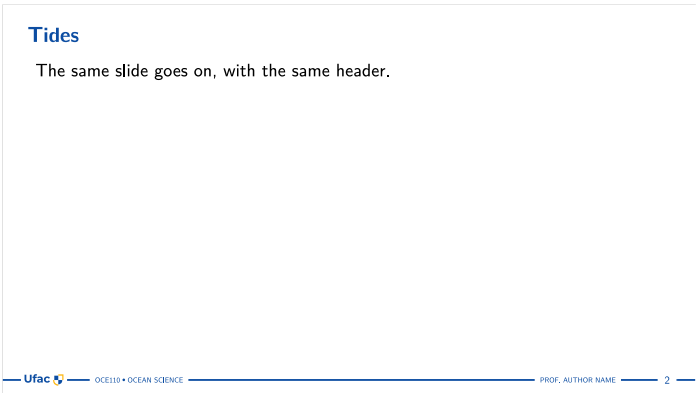
```
== Tides

The sea rises and falls about
twice a day, pulled by the Moon
and the Sun.

---

The same slide goes on, with
the same header.
```





> ⚠ **Caution!** Touying breaks the page at *any* top-level em dash, so an em dash next to inline markup, as in `Text — *bold*`, also starts a new page. Use an en dash or wrap the paragraph in `#[ ]`.

The `===` subtitle is repeated on the pages that continue the slide, unless the continuation starts with another `===`; that holds for `---`, `#pagebreak()` and a content that overflows. Only a top-level `===` is repeated: inside `#[ ]`, a column or a box, the pill stays in the flow.

```
== Sea creatures

=== Octopus

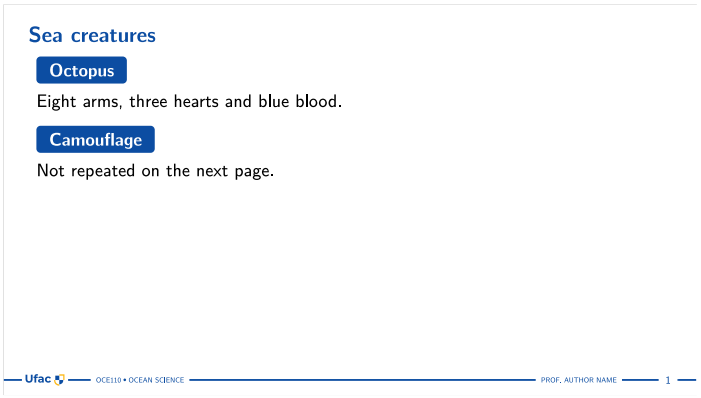
Eight arms, three hearts and
blue blood.

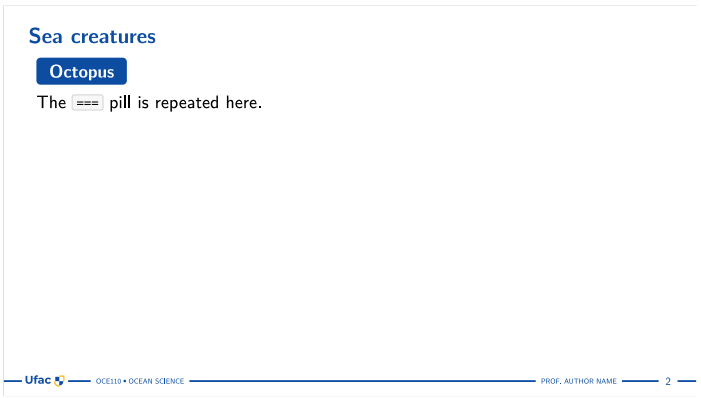
==== Camouflage

Not repeated on the next page.

---

The `===` pill is repeated here.
```

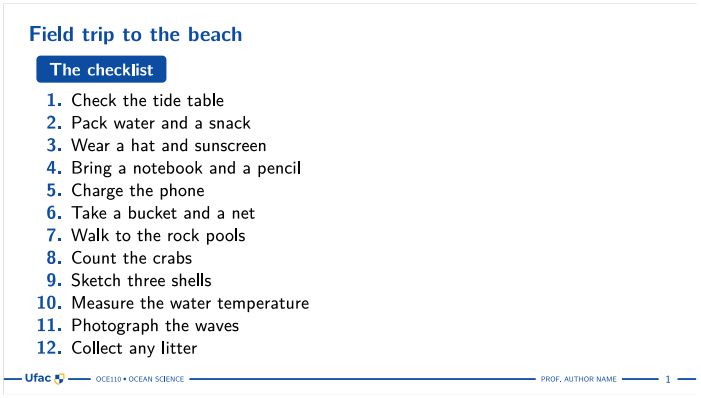


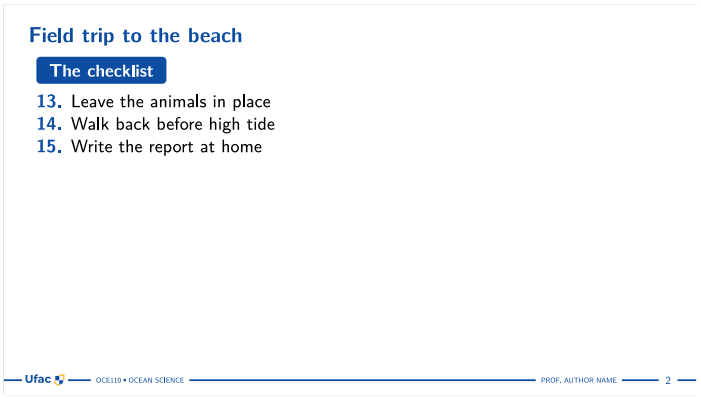



```
== Field trip to the beach

=== The checklist

+ Check the tide table
+ Pack water and a snack
+ Wear a hat and sunscreen
+ Bring a notebook and a pencil
+ Charge the phone
+ Take a bucket and a net
+ Walk to the rock pools
+ Count the crabs
+ Sketch three shells
+ Measure the water temperature
+ Photograph the waves
+ Collect any litter
+ Leave the animals in place
+ Walk back before high tide
+ Write the report at home
```



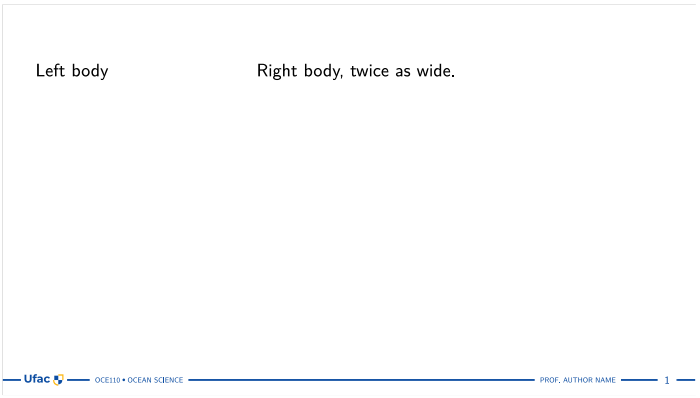


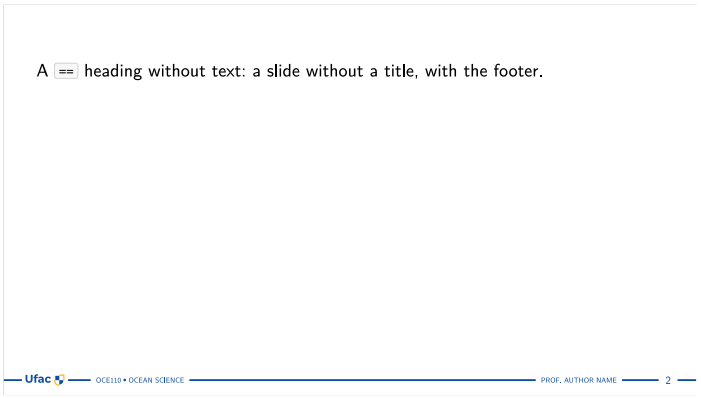
`#slide` is the slide that every `==` opens; calling it gives Touying's arguments, such as `composer`, which lays out the bodies of a slide in columns.

Reference: `slide()` p. 34

```
#slide(composer: (1fr, 2fr))[
  Left body
][
  Right body, twice as wide.
]

==
A `==` heading without text: a
slide without a title, with the
footer.
```





5 Exercises and examples

A label on the `==` heading turns the slide into the box of an exercise (red) or an example (green), as in a printed handout. The box fills the slide; the title goes on a badge over its top border, with the icon of the kind (a pencil, a beaker), the name, the number and the title; the footer of the slide closes the box. A suffix makes the label unique, as in `<exercise-tides>`, so the slide can be cited: `@exercise-tides` becomes "Exercise 1.N", linked to the slide.

Exercises and examples are numbered on their own, after a prefix, `config-info(counter-prefix:)`: `[1.]` by default gives "Exercise 1.1", "Exercise 1.2"; `[3.]`, for the third unit of a course, gives "Exercise 3.1"; `none` gives "Exercise 1", the default of the kinds without units (Section 2.2).

```
#show: ufac-theme.with(config-info(
  subject: [Ocean Science],
  counter-prefix: [3.],   // "Exercise 3.1"; none gives "Exercise 1"
))
```

Inside the box, what has no style of its own takes the color of the item: the bold text, the markers, the arrows, the numbers of the lists, the inline code, the links and the `===` pills. A code block, a callout and an `emph-box` keep their own look.

`#solution` (in an example) and `#answer` (in an exercise), alone on a line like `#pause`, open the solution: a dashed line and "Solution." or "Answer." in bold at the start of the next paragraph; after a blank line, the word stays alone and the solution starts a new paragraph. With a `#pause` right before or after the marker, the solution appears on the next step of the slide (Section 10.1). The word ends with a period; `#answer(sep: ":")` ends it with a colon, and `#solution(sep: [:\ ])` with a colon and a line break, so that the solution starts on the next line.

An exercise or example goes on until a slide of another kind: a long content continues on the next page in the same box, with the same number, and `---` or `#pagebreak()` breaks the page by hand. `#exercise-slide` and `#example-slide` are the function forms, with a color and an icon of their own; they cannot be cited.

Reference: `exercise-slide()` p. 36, `example-slide()` p. 37, `solution()` p. 38, `answer()` p. 38

```
== Tide table <exercise-tides>

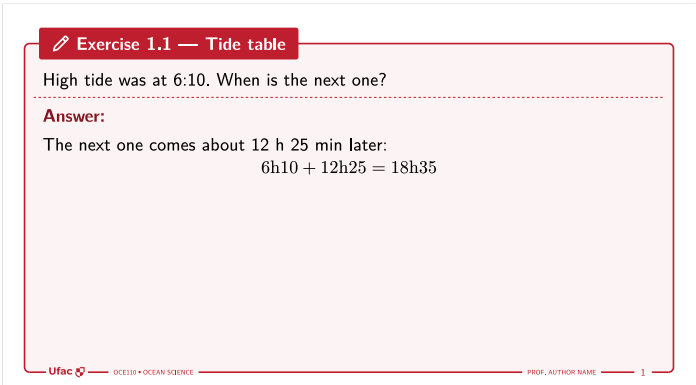
High tide was at 6:10. When is
the next one?

#answer(sep: ":")

The next one comes about
12 h 25 min later:

$ 6"h"10 + 12"h"25 = 18"h"35 $

== Wave speed <example>
```



```
Is a long wave faster than a
short one in deep water?

#pause
#solution
Yes: the speed grows with the
square root of the wavelength.

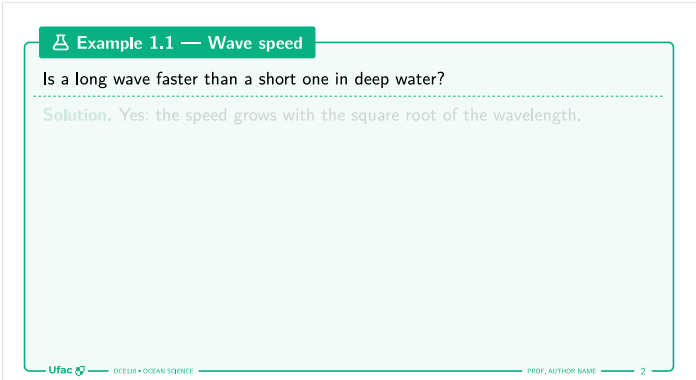
#exercise-slide(
  title: [Field trip],
  color: colors.primary,
  icon: "terminal",
)[
  The function form, with another
  color and icon: how long
  between two high tides?

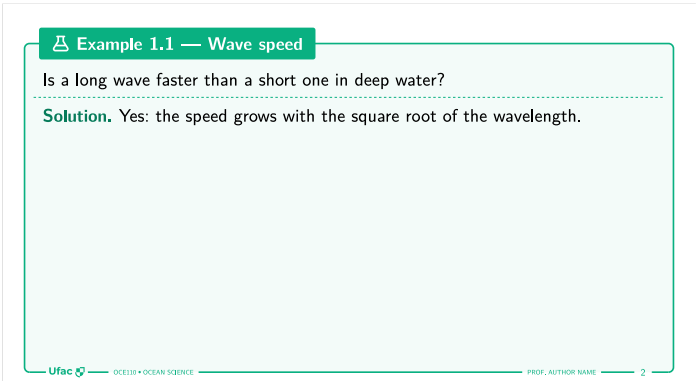
  #answer
  About 12 h 25 min.
]

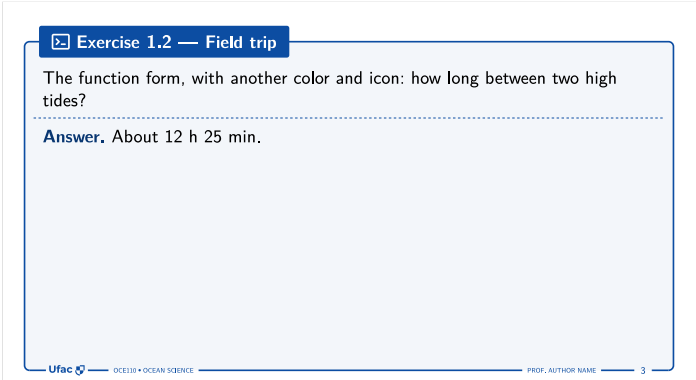
#example-slide(
  title: [Lighthouse],
  icon: none,
)[
  Without icon: `icon: none`.
]

== Review

Solve @exercise-tides before
the next class.
```









Review

Solve **Exercise 1.1** before the next class.

6 Writing on a slide

Most emphases have two forms: a function and a **shorthand**, a short piece of markup that the theme rewrites into the same function. The shorthands take plain text only (no markup inside), and none of them applies inside `raw`.

Function	Shorthand	Result
`#strong[text]`	`*text*`	black bold
`#emph[text]`	`_text_`	black italic
`#alert[text]`, `#primary[text]`	`~~text~~`	blue bold
`#highlight[text]`	`==text==`	light-yellow background
`#underline[text]`	`~text~`	strong yellow underline
`#secondary`, `#tertiary`, `#quaternary`		bold in yellow, red, green
`#arrows[..]`	`-> item`	arrow list
`#emph-box[..]`	`> text`	box (Section 7)
`$ #eq-box[$..$] $`	`> $ .. $`	equation box (Section 7)
`#quote-box[..]`	`"text"`	quotation (Section 7)

6.1 Emphasis

```
Black *bold* and _italic_.

Blue #alert[alert], the same as
~~this shorthand~~.

#tertiary[Problem:] a colored label.
```

Black **bold** and *italic*.

Blue **alert**, the same as **this shorthand**.

Problem: a colored label.

The `==text==` shorthand cannot open a line (there it is a heading), and its text neither starts nor ends with a space; `#highlight` has no such limits and takes another fill, such as a `-lighter` tone of the palette (Section 9.2).

```
A ==highlighted== word and a
#highlight(
  fill: colors.quaternary-lighter,
)[green one].

A ~strong underline~ and
#underline[the function form].
```

A highlighted word and a green one.

A strong underline and the function form.

Reference: `primary()` p. 44, `secondary()` p. 45, `tertiary()` p. 45, `quaternary()` p. 45

6.2 Code chips

Inline `raw` is a gray chip. Inside a title or a box it follows the color of the container, and inside a pill it is inverted.

```
Call `fit(x, y)` first.

#emph-box(color: colors.tertiary)[
  Never call `predict` before `fit`.
]
```

Call `fit(x, y)` first.

Never call `predict` before `fit`.

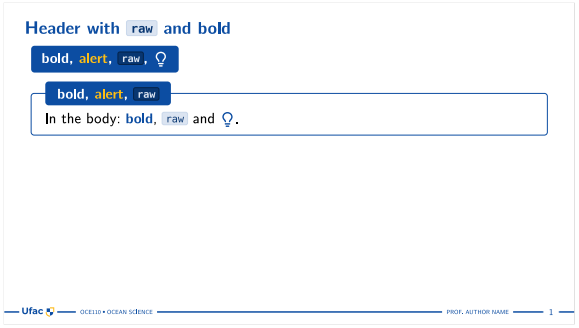
6.3 Colors by context

Bold text, alerts, highlights, underlines, arrows, chips, links, icons (Section 10.3) and the `===` pills read the color of what contains them: the color of an `emph-box`, a callout, an exercise or an example, or of a light cell of a table (Section 8.1); white or yellow inside a pill, where the color of the pill would hide them. What has a style of its own keeps it: a code block, a box inside another box, which brings its own color, and a reference to an exercise or example, in the color of its kind.

```
== Header with `raw` and *bold*

=== *bold*, ~~alert~~, `raw`, #icon("tip")

#emph-box(
  title: [*bold*, ~~alert~~, `raw`],
)[
  In the body: *bold*, `raw`
  and #icon("tip").
]
```



6.4 Lists and terms

Consecutive lines starting with `->` become one list with a blue arrow. The list has to start the paragraph, after a blank line; an arrow in the middle of a line and the math arrow are left alone. Bullet lists have three levels of yellow markers, and numbered lists are blue and bold: `1.`, `a)`, `i.` by level. `#set enum(numbering: "a)")` starts at `a)`; Typst only knows `+` and `1.` as markers, so `a)` at the start of a line is text.

```
-> consequence
-> another consequence

- first level
  - second level
    - third level

+ first
  + nested

/ Tide: the rise and fall of the sea.
```

→ consequence
→ another consequence

- first level
  - second level
    - third level

1. first
   a) nested

Tide: the rise and fall of the sea.

Reference: `arrows()` p. 45

6.5 Columns

`#cols` puts its bodies side by side anywhere in a slide, with an optional divider; `#slide(composer:)` lays out a whole slide (Section 4.3).

```
#cols[Left][Middle][Right]
```

Left Middle Right

Reference: `cols()` p. 46

6.6 Escapes

A backslash on the sign keeps a shorthand literal. An escaped `>` is only recognized when a word follows it.

```
\==not highlighted== and
\~not underlined\~

\> not a box

-\> not a list

\"not a quotation\"
```

==not highlighted== and ~not underlined~

> not a box

-> not a list

"not a quotation"

7 Boxes

7.1 Emphasis boxes

`#emph-box` frames a definition or a key idea: a thin border in a color of the palette, and an optional title on a badge over the border, with an optional icon.

```
#emph-box[
  A definition, in the primary color.
]

#emph-box(color: colors.tertiary,
  title: [Problem])[
  The anchor *drags* and the boat
  ~drifts~.
]

#emph-box(color: colors.quaternary,
  icon: "tip", title: [Tip])[
  An icon before the title.
]
```

A definition, in the primary color.

Problem

The anchor **drags** and the boat ~~drifts~~.

Tip

An icon before the title.

A paragraph whose lines start with `>` is the same box in the primary color; a first line `> == Title` gives the title. Only inline content fits there: lists and code blocks need `#emph-box`. A loose `>` at the start of a paragraph is read as a box: write `$a > b$`, or escape it (Section 6.6).

```
> A box without title, with *bold*
> and $x^2$.

> == Definition
> A box with a title.
```

A box without title, with **bold** and x^2.

Definition

A box with a title.

Reference: `emph-box()` p. 39

7.2 Equation boxes

`#eq-box` frames a key equation from inside the equation, with dollar signs around its body; `>` before a block equation, on the same line or the line above, does the same.

```
$ #eq-box[$E = m c^2$] $

> $ c = sqrt(g h) $
```

$$E = mc^2$$

$$c = \sqrt{gh}$$

Reference: `eq-box()` p. 40

7.3 Callouts

A callout is a box with the color, the icon and the title of its kind, the title in the language of the deck. Each kind has a role and a function of its own: `#caution`, a mistake to avoid; `#tip`, a shortcut; `#remark`, titled "Reminder" ("Relembrando" in Portuguese), to recall what was seen before; `#note`, something secondary. The default style is the box of a printed handout, with a bar on the left and the title opening the text; `style: "badge"` gives an emphasis box with the title on a badge.

```
#caution[
  `x*y` with two lists is an error.
]
#tip[The up arrow recalls a command.]
#remark[
  A *function* is a named block of code.
]
#note[Two versions of the program
  differ here.]
#callout("tip", style: "badge")[
  The badge style.
]
```

Caution! `x*y` with two lists is an error.

Tip. The up arrow recalls a command.

Reminder. A **function** is a named block of code.

Note. Two versions of the program differ here.

Tip

The badge style.

Reference: `callout()` p. 41, `caution()` p. 42, `tip()` p. 42, `remark()` p. 42, `note()` p. 43

7.4 Tags and quotations

`#tag` is the yellow tag of the cover and of the section slides, for the text, the titles and the tables: a short label in spaced capitals, on any color. A paragraph made only of quoted lines becomes one quotation per line, a `#quote-box`; Typst's `#quote(attribution:)` is a `#quote-box` too, with the attribution on the right.

```
#tag[Optional]
#tag(color: colors.tertiary)[Hard]

"All models are wrong"
"but some are useful"

#quote(attribution: [George Box])[
  All models are wrong.
]
```

OPTIONAL HARD

All models are wrong

but some are useful

All models are wrong.

— George Box

Reference: `tag()` p. 43, `quote-box()` p. 44

8 Tables, figures, math and code

8.1 Tables

The header row is blue with white text. A cell filled with a light tone of the palette (`table.cell(fill: colors.secondary-lighter)`) colors its chips and bold text like a box of that family; a cell filled with a dark tone is like a pill, with white text.

```
#table(
  columns: 3,
  table.header(
    [Fish], [Depth (m)], [$T$ (°C)],
  ),
  [Cod], [200], [4],
  [Tuna], [50], table.cell(
    fill: colors.secondary-lighter,
  )[*18*],
)
```

Fish	Depth (m)	T (°C)
Cod	200	4
Tuna	50	**18**

8.2 Figures and math

A figure caption shows its body only, without "Figure N". Math uses New Computer Modern Math, and block equations sit closer to the text than Typst's default.

```
#figure(
  rect(width: 60%, height: 3em,
    fill: colors.primary-lightest),
  caption: [The caption, no number.],
)

The speed of a wave in deep water is
$ c = sqrt((g lambda) / (2 pi)) $
```


The caption, no number.

The speed of a wave in deep water is

$$c = \sqrt{\frac{g\lambda}{2\pi}}$$

8.3 Code

A code block goes in the box of the theme: a light blue fill with a bar on the left, the line numbers (a long line wraps and continues under the code), the language in a pill at the top right and the syntax colors of the theme. The output of a program, a block in `text` (or `txt`, `output`), goes in a gray box without numbers. Inside a colored box a block keeps its look. The languages with a pill are `python` (`py`), `typst` (`typ`), `matlab`, `bash` and `sh`, and the output has its own ("Output"); `ufac-theme(code-languages:)` adds others.

A long block goes on to the next page of the slide, as any long content, with the `===` pill repeated at the top; the language pill appears only on its first part. A callout does not split, so a long listing goes outside it. For a file name, highlights or ranges, the deck takes codly (Section 10.2).

Reference: `code-languages` p. 47, `ufac-syntax` p. 47

````
== Code blocks

```python
def scale(grams, people, n=4):
    return grams * people / n
```

```text
375.0
```

```bash
typst compile slides.typ
```
````



9 Customization

9.1 Language

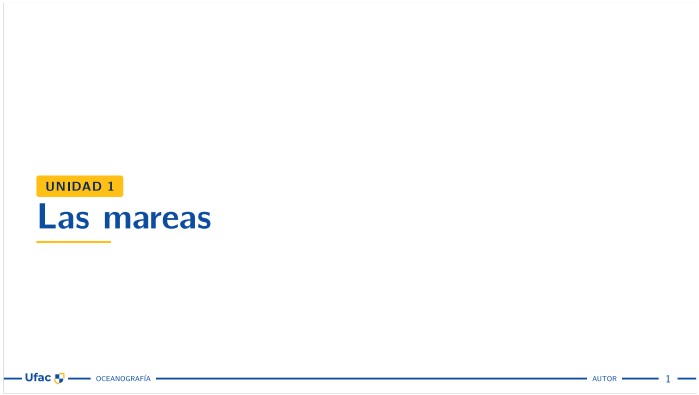
The theme speaks Portuguese by default (`lang: "pt-br"`); `lang: "en"` or `"es"` switches the language of the text and every word of the theme: the names of exercises, examples and parts, the titles of the callouts, “Solution” and “Answer”, “Output”, “Thank you!”, the dates and the tags of the cover. The names of exercises, examples and parts can also be given one by one.

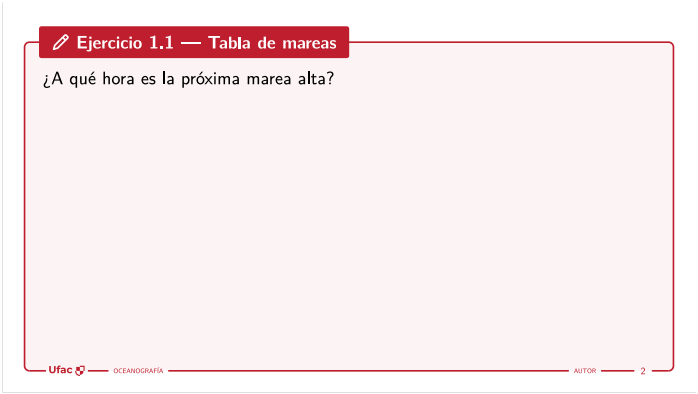
```
#show: ufac-theme.with(
  lang: "es",
  part-name: [Unidad],
  config-info(
    subject: [Oceanografía],
    author: [Autor],
  ),
)

= Las mareas

== Tabla de mareas <exercise>

¿A qué hora es la próxima
marea alta?
```





9.2 Colors

Five families of seven tones, named after Touying’s colors. The light tones serve as fills of highlights and boxes, the dark ones as text on white; the tones are derived from the base color, and the neutral family goes from black to white.

	`-darkest`	`-darker`	`-dark`	base	`-light`	`-lighter`	`-lightest`
`primary`	#031329	#052349	#083671	#0C4DA2	#5582BE	#9EB8DA	#DBE4F1
`secondary`	#403005	#735609	#B2860E	#FFBF14	#FFD25A	#FFE5A1	#FFF5DC
`tertiary`	#30070B	#560D14	#851520	#BE1E2D	#D2626C	#E5A5AB	#F5DDE0
`quaternary`	#022C20	#044F3A	#067B5A	#09B081	#53C8A7	#9DDFCD	#DAF3EC
`neutral`	#000000	#2B2B2B	#555555	#808080	#AAAAAA	#D5D5D5	#FFFFFF

```
A #highlight(
  fill: colors.tertiary-lighter,
)[red fill] and
#text(fill: colors.primary-dark)[dark
blue text]
```

A red fill and dark blue text

Reference: colors p. 48

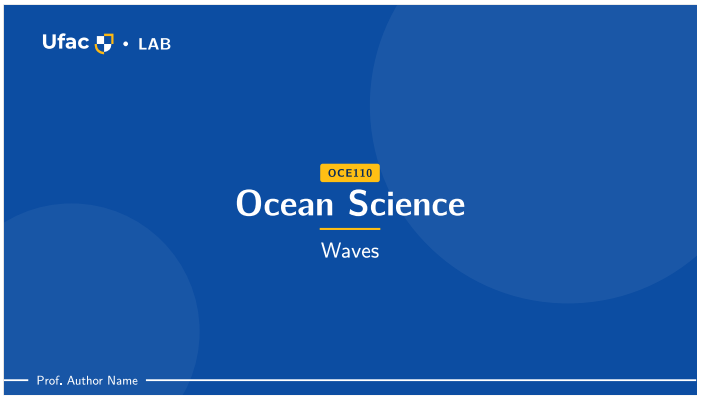
9.3 Logos

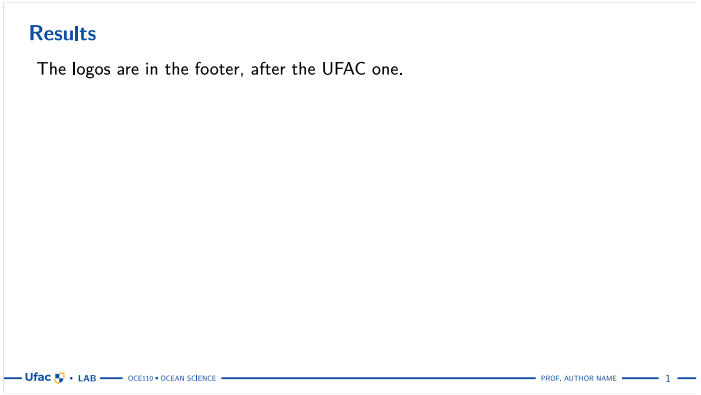
The footer and the cover show the UFAC logo. `config-info(logo:)` is the list of the logos shown there, separated by dots: `(auto, x)` keeps the UFAC logo and adds `x` after it, `(x,)` replaces it and `none` removes them. A logo is content, or a function of the color of the footer (blue on the slides, white on the cover), so that one drawing serves both; about `0.64em` high, it matches the UFAC logo. In a slide, `#info("logo")` shows them (Section 9.5); outside a deck, as in a handout, `image(bytes(ufac-logo))` draws the UFAC logo.

```
#show: ufac-theme.with(
  lang: "en",
  config-info(
    logo: (auto, color => text(
      size: .5em, fill: color,
      weight: "bold")[LAB]),
    subject: [Ocean Science],
    subject-code: [OCE110],
    unit-title: [Waves],
    author: [Prof. Author Name],
  ),
)

#title-slide()

== Results
The logos are in the footer,
after the UFAC one.
```





9.4 The footer and the badges

The footer shows the short title and the title on the left and the authors on the right, each on one line: when the two do not fit, they are cut with "…" at the end of a word (the authors keep at least a third of the room). `ufac-theme(footer-left:, footer-right:)` replaces either with content or a function of `self`. The badge of the exercises and examples takes another name (`exercise-name`, `example-name`) and another icon (`exercise-icon`, `example-icon`), and the section slides another word (`part-name`).

```
#show: ufac-theme.with(
  footer-left: [Introductory Astronomy],
  exercise-name: [Task], exercise-icon: "terminal",
  config-info(subject: [Astronomy]),
)
```

Reference: `ufac-theme()` p. 31

9.5 The information anywhere: `#info`

`#info` shows any field of the deck, or any alias, anywhere in a slide, inside boxes and tables too: the author as written, the date in full, the contact, and with `"logo"` the logos of the footer, with a height and in white for a dark background. With a function, it receives every field at once.

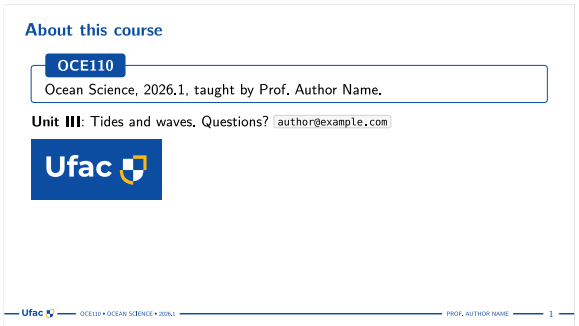
Reference: `info()` p. 38

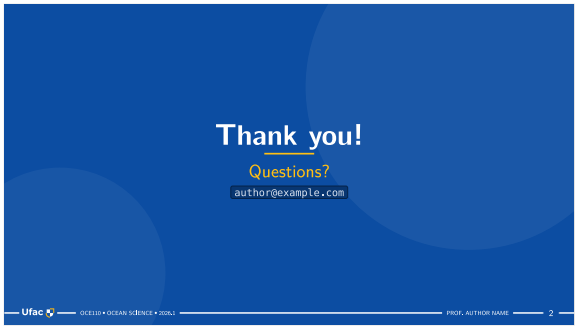
```
#show: ufac-theme.with(
  lang: "en",
  config-info(
    subject: [Ocean Science],
    subject-code: [OCE110],
    unit-title: [Tides and waves],
    unit: [Unit III],
    author: [Prof. Author Name],
    semester: [2026.1],
    contact: [`author@example.com`],
  ),
)

== About this course
#emph-box(
  title: info("subject-code"),
)[
  #info("subject"), #info("semester"),
  taught by #info("author").
]
#info(i => [
  *#i.unit*: #i.unit-title.
])
Questions? #info("contact")

#box(fill: colors.primary,
  inset: 0.4em, info("logo",
    height: 2em, white: true))

#ending-slide(
  subtitle: [Questions?],
)[#info("contact")]
```





10 Integrations

The theme sits on Touying and works with the packages a deck usually takes along: codly for the code blocks, and the icons of any collection of Iconify.

10.1 Touying

The theme leaves the features of Touying alone. Animations work on every element of the theme: covered content stays faintly visible under a veil in the color of the page.

```
== Animations

First this.

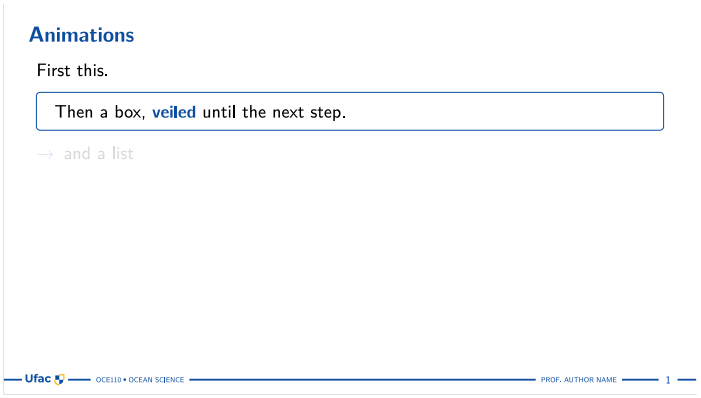
#pause

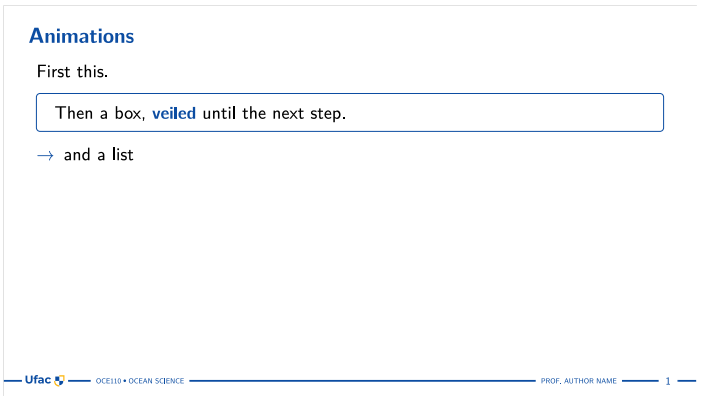
> Then a box, *veiled*
> until the next step.

#pause

-> and a list
```







⚠ **Caution!** Keep `#pause` and `#uncover` / `#only` in separate paragraphs: within one paragraph Touying reorders the covered part on the first subslide.

Feature	Functions
Animations	`#pause`, `#meanwhile`, `#jump`, `#uncover`, `#only`, `#alternatives`, `#item-by-item`, `#effect`, `#touying-raw`
Notes and handouts	`#speaker-note` (it attaches to the previous slide), `#handout-only`, `config-common(handout: true)`
Layout	`#slide(composer: (1fr, 2fr))`, `#cols`, `components.adaptive-columns`, `components.full-width-block`, `utils.fit-to-width`, `utils.fit-to-height`
Overflow	`config-common(breakable: false, clip:, detect-overflow:)`
Reuse	`#touying-recall`, `touying-get-config`, `components.progressive-outline`

The packages Touying integrates work as in any Touying theme: CeTZ and Fletcher, whose diagrams reveal their parts with `#pause` through `touying-reduce`; pinit, for pointing at parts of a slide; and MiTeX, for LaTeX formulas. Their pages in the documentation of Touying show how. Instead of the `outline-slide` of other Touying themes, write a slide with `#outline()`; `focus-slide` and `matrix-slide` have no counterpart here.

10.2 Code blocks with codly

The theme draws its own code box (Section 8.3) and does not need codly. For codly's features (a file name, highlights, ranges, `diff`), the deck imports codly 2 and passes it to the theme, `ufac-theme(codly: codly)`: every block goes through codly with the same look, and codly's functions (`codly.new(..)`, `codly.set_(..)`) are at hand. codly 2 requires Typst 0.15. The languages of the codly-languages package can be given to `ufac-theme(code-languages:)`.

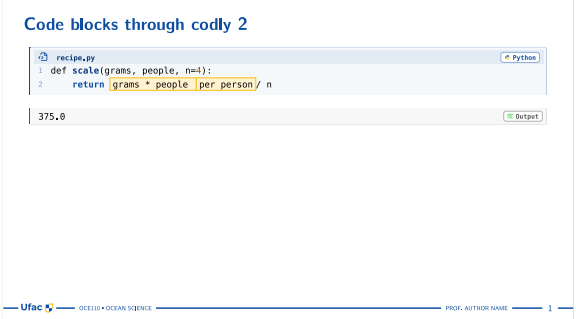
Reference: codly-ufac-theme p. 47

````
#import "@preview/codly:2.0.0" as codly
#show: ufac-theme.with(
  lang: "en",
  codly: codly,
  config-info(
    subject: [Ocean Science],
    subject-code: [OCE110],
    unit-title: [Tides and waves],
    unit: [Unit III],
    author: [Prof. Author Name],
  ),
)

== Code blocks through codly 2

#codly.new(
  ```python
  def scale(grams, people, n=4):
      return grams * people / n
  ```,
  file: "recipe.py",
  highlights: (
    (line: 2, start: 12, end: 26,
      tag: [per person]),
  ),
)

```text
375.0
```
````



10.3 Icons

`#icon` draws an icon on the line of text, in the color of its surroundings: black in the text, the color of the box inside a box, white in a pill. The theme ships a set of icons, each with a role that is also its name for `#icon`; the colored ones keep their colors.

Reference: `icon()` p. 47

```
#emph-box(color: colors.tertiary)[
  #icon("caution") takes the
  color of the box.
]
```

takes the color of the box.

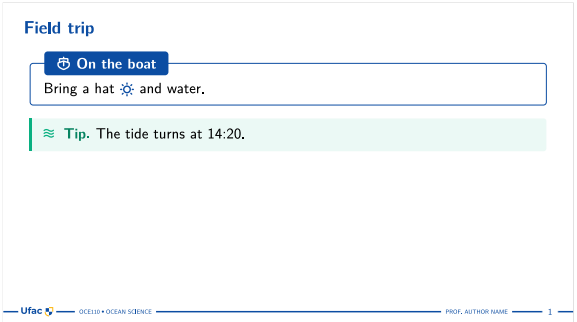
warning (octicon:alert-16) · caution (octicon:alert-16) · tip (octicon:light-bulb-16) · info (octicon:info-16)
note (octicon:note-16) · remark (octicon:history-16) · exercise (octicon:pencil-16) · example (octicon:beaker-16)
file (octicon:file-code-16) · terminal (octicon:terminal-16) · contact (octicon:mail-16) · output (vscode-icons:file-type-log)
code (ph:code-bold) · python (devicon:python) · matlab (devicon:matlab) · shell (material-icon-theme:console)
typst (simple-icons:typst)

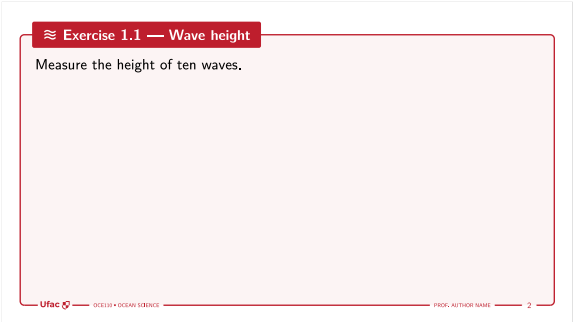
`"warning"` and `"caution"` are the same icon, under the name of 0.2.0 and the one of the callout. The Octicons the theme ships also answer to their own names: `"alert"`, `"light-bulb"`, `"info"`, `"note"`, `"history"`, `"pencil"`, `"beaker"`, `"file-code"`, `"terminal"` and `"mail"`. The icons come from Octicons (© GitHub Inc., MIT license), Phosphor (MIT license), Devicon (MIT license), Simple Icons (CC0), Material Icon Theme (MIT license) and VSCode Icons (MIT license).

Any other icon comes from a collection in the JSON format of Iconify, read by the iconify package, which the theme imports: the deck gives the collection to `provide-icons` and writes the icon as `"collection:name"`, in `#icon` or wherever the theme takes the name of an icon (an `emph-box`, a callout, the badge of an exercise or example). It is then an icon like those of the theme, with the same size, the same place on the line and the color of its surroundings.

```
== Field trip
#provide-icons(json("ph.json"))
#emph-box(icon: "ph:boat-bold",
  title: [On the boat])[
  Bring a hat #icon("ph:sun-bold")
  and water.
]
#tip(icon: "ph:waves-bold")[
  The tide turns at 14:20.
]

#exercise-slide(
  title: [Wave height],
  icon: "ph:waves-bold",
)[
  Measure the height of ten waves.
]
```





There are two ways to get a collection; the names of its icons are on icones.js.org, and each collection has its own license.

- **Only the chosen icons**: ask the Iconify API for them and save the answer, here as `ph.json`: `https://api.iconify.design/ph.json?icons=boat-bold,sun-bold,waves-bold`.
- **The whole collection**: download its JSON from the site of the iconify package (some are several megabytes).

11 Moving from 0.2.0

A deck of 0.2.0 needs these changes; everything else keeps working.

In 0.2.0	In 0.3.0
`config-info` with the fields of the theme: `subject`, `subject-code`, `title` (the topic), `subtitle` (the unit)	keep `subject` and `subject-code`, now aliases of Touying's `title` and `short-title`; rename the old `title` to `unit-title` and the old `subtitle` to `unit` (Section 2). `subject` and `title` together are an error. A function of `footer-left` or `footer-right` reads Touying's names, `self.info.title` and `self.info.short-title`, or the footer is content with `#info("subject")`
`ufac-theme(logos: ..)`	`config-info(logo: ..)` (Section 9.3)
`ufac-theme(section-margin: ..)`; the figure of a section flush with the edge	the figure keeps the page margin; for the old look, `config-common(new-section-slide-fn: new-section-slide.with(margin: 0pt))`
`<empty-slide>`, `<empty-slide-COLOR>` and the theme's `#empty-slide`	removed: the theme has no empty slide of its own (below)
`local`, `codly-range`, `no-codly`; codly imported by the theme; `#local(breakable: true)` for a block that splits	the theme has its own code box, which goes on to the next page by itself (no `breakable:` needed); for codly, `#import "@preview/codly:2.0.0" as codly` and `ufac-theme(codly: codly)`
`#icon("rocket")` and the other Octicons	only the icons of the theme; another one comes from its collection: `#provide-icons(json("octicon.json"))` and `#icon("octicon:rocket-16")` (Section 10.3)
`ufac-logo-image(..)`	`#info("logo", height: 2em)`, with `white: true` on a dark background; outside a deck, `image(bytes(ufac-logo))`

A label `<empty-slide…>` of 0.2.0 is now an ordinary label, so its slide becomes an ordinary one, with its header: look for these labels in the deck. The blue page of the cover, with the footer, is `#ending-slide(title: none)[..]` (Section 3); a page without header or footer is Touying's `#empty-slide[..]` (the deck imports Touying), which takes neither `color:` nor `title:`, so a page of another color sets its fill and the color of its text:

```
#empty-slide(config: config-page(fill: colors.tertiary))[
  #set text(fill: white)
  #align(center + horizon)[*Break*: 15 minutes]
]
```

A `==` without text gives a slide without a title, with the footer.

The figure of a section, which 0.2.0 cropped to fill 12em × 66% of the slide, now fits whole in that frame; a deck needs no change for it.

12 Reference

Every function and variable of the theme, with the type of each parameter, its default and its description. The chapters above show how to use them.

12.1 The theme and the slides

`ufac-theme()`

```
ufac-theme(
  aspect-ratio: "16-9"  str
  lang: "pt-br"  str
  exercise-name: auto  auto  content
  example-name: auto  auto  content
  exercise-icon: "pencil"  none  str  content  symbol
  example-icon: "beaker"  none  str  content  symbol
  part-name: auto  auto  content
  footer-left: auto  auto  content  function
  footer-right: auto  auto  content  function
  background: "circles"  str
  codly: none  none  module
  code-languages: (:)  dictionary
  ..args  dictionary
  body  content
) -> content
```

The theme. It sets the page (22pt New Computer Modern Sans, the UFAC header and footer) and every rule of the syntax, and takes Touying's configs as extra arguments.

```
#show: ufac-theme.with(
  aspect-ratio: "16-9",
  config-info(
    subject: [Ocean Science], subject-code: [OCE110],
    unit-title: [Tides and waves], unit: [Unit III],
    author: [Prof. Author Name], counter-prefix: [1.],
  ),
)
```

`config-info` carries the fields of Touying: `title` (the name of the course or of the work), `short-title` (its code or a shorter name: the tag of the cover), `subtitle` (the topic), `short-subtitle` (the lecture or the unit), `author` (one, an array or free content), `date`, `institution`, `logo` and `contact`; the aliases of a course (`subject`, `subject-code`, `unit`, `unit-title`, `course`, `semester`, `year`); the `kind` of presentation; and `counter-prefix`, the prefix of the exercise and example numbers ("Exercise 1.N"; `none` gives "Exercise N", the default of the kinds without units: `"coursework"`, `"talk"`, `"tcc"`, `"dissertation"`, `"thesis"`).

Argument

`aspect-ratio: "16-9"` `str`

Any ratio accepted by Touying's `utils.page-args-from-aspect-ratio`: `"16-9"`, `"4-3"`, …

Argument

`lang: "pt-br"` `str`

Language of the deck: `"pt-br"` (or `"pt"`), `"en"` or `"es"`, optionally with a region (`"en-US"`). It sets `text.lang`/`region` and the localized names below.

Argument

`exercise-name: auto` — `auto` | `content`

Name on the badge of exercise slides; `auto` is Exercício / Exercise / Ejercicio.

Argument

`example-name: auto` — `auto` | `content`

Name on the badge of example slides; `auto` is Exemplo / Example / Ejemplo.

Argument

`exercise-icon: "pencil"` — `none` | `str` | `content` | `symbol`

Icon before the name on the badge of exercise slides: the name of an `icon`, any content (an image or a symbol, centered on the capitals like the icons) or `none` for no icon. `exercise-slide(icon:)` changes it on one slide.

Argument

`example-icon: "beaker"` — `none` | `str` | `content` | `symbol`

Icon before the name on the badge of example slides, like `exercise-icon`.

Argument

`part-name: auto` — `auto` | `content`

The word of the tag of the section slides ("PART N"); `auto` is Parte / Part / Parte.

Argument

`footer-left: auto` — `auto` | `content` | `function`

Left text of the footer; a function receives `self`. `auto` is the `short-title` and the `title` separated by a dot and, when the date is a `semester` or a `year`, the date too ("AST101 ▪ Astronomy ▪ 2026.2").

Argument

`footer-right: auto` — `auto` | `content` | `function`

Right text of the footer; a function receives `self`. `auto` is the authors in running text ("A, B and C"); of an author written as free content, the names of its first group.

Argument

`background: "circles"` — `str`

Background of the cover and of the closing slide: `"circles"` (the solid UFAC blue with two translucent circles), `"gradient"` (a blue gradient, with the circles) or `"plain"` (the blue alone).

Argument

`codly: none` — `none` | `module`

The codly 2 module, when the deck imports it (`#import "@preview/codly:2.0.0" as codly`): the code blocks then go through codly, with the look of the theme (`codly-ufac-theme`). `none` gives the code box of the theme, without codly.

Argument

`code-languages: (:)` — `dictionary`

Languages added to those of the theme (`code-languages`) or replacing any of them except the output, as `(name:, icon:)`.

Argument

`..args` `dictionary`

Touying configs, typically `config-info(..)`; also `config-common(..)`, `config-page(..)`.

Argument

body `content`

The deck.

`title-slide()`

```
title-slide(
  style: auto  auto  str
  author-label: auto  auto  none  content
  header-left: auto  auto  none  content  function
  header-center: auto  auto  none  content  function
  header-right: auto  auto  none  content  function
  footer-left: auto  auto  none  content  function
  footer-center: auto  auto  none  content  function
  footer-right: auto  auto  none  content  function
  footer-line: auto  auto  bool
  ..body  content
) -> content
```

Cover: the blue page (`ufac-theme(background:)`) with the logos large at the top left; in the center, the tag (`short-title`), the `title`, the yellow rule, the `short-subtitle` in yellow, the `subtitle` and the content given to it, centered both ways in the room between the top and the bottom; the authors and the date where the `style` puts them. The institution does not appear (it is for `#info`). The slide counter is frozen, so the cover does not count.

Each position takes `auto` (what the style puts there), `none` (nothing), content (such as `#info("date")`) or a function of `self`, like `footer-left` of the theme. The text of the positions is 0.7 of the size of the slide.

```
#title-slide()
#title-slide(style: "top", footer-center: none, footer-right: info("date"))
#title-slide[Slides at #link("https://example.com/slides")[example.com/slides]]
```

Argument

`style: auto` `auto` | `str`

The logos are at the top left in every style. `"line"`: the authors on one line at the bottom left, the white line and the date on the right. `"stack"`: the authors on several lines at the bottom left and the date on the right, without the line. `"top"`: the authors at the top right and the date in the center of the bottom, without the line. `auto` is the one of the `kind` of the deck, or `"line"`.

Argument

`author-label: auto` `auto` | `none` | `content`

Label over the authors (“Instructors”); before them in `"line"`. `auto` is the one of the `kind`, or none.

Argument

`header-left: auto` `auto` | `none` | `content` | `function`

What goes at the top left.

Argument

`header-center: auto` — `auto` | `none` | `content` | `function`

What goes at the top center.

Argument

`header-right: auto` — `auto` | `none` | `content` | `function`

What goes at the top right.

Argument

`footer-left: auto` — `auto` | `none` | `content` | `function`

What goes at the bottom left.

Argument

`footer-center: auto` — `auto` | `none` | `content` | `function`

What goes at the bottom center.

Argument

`footer-right: auto` — `auto` | `none` | `content` | `function`

What goes at the bottom right.

Argument

`footer-line: auto` — `auto` | `bool`

The white line at the bottom, like the one of the footer of the slides: from edge to edge, passing between what is at the bottom, at the height of the line of the footer. `auto` is the one of the style.

Argument

`..body` — `content`

Content in the center, below the titles, `#title-slide[..]`; `#title-slide()` has none.

`slide()`

```
slide(
  config: (:) dictionary
  repeat: auto auto int
  setting: body => body function
  composer: auto auto array function
  ..bodies content
) -> content
```

Default slide of the theme, with header and footer. Every `==` heading opens one, so it is rarely called by hand; the explicit form gives access to Touying's arguments.

A top-level `===` subtitle is repeated on the continuation pages of the slide. A slide without a title is a `==` heading without text.

```
#slide(composer: (1fr, 2fr))[Left][Right]
```

Argument

`config: (:)` — `dictionary`

Slide configuration: a `config-xxx` dictionary (merge several with `utils.merge-dicts`).

Argument

`repeat: auto` — `auto` | `int`

Number of subslides; `auto` lets Touying count the animations.

Argument

`setting: body => body` — `function`

Set/show rules applied to the slide body.

Argument

`composer: auto` — `auto` | `array` | `function`

Layout of the bodies, e.g. `(1fr, 2fr)`, as in Touying's `side-by-side`.

Argument

`..bodies` — `content`

The slide bodies.

new-section-slide()

```
new-section-slide(
  config: (:) dictionary
  level: 1 int
  numbered: true bool
  margin: auto auto length
  body content
) -> content
```

Section slide, created by every `= Title`: on the left, "PART N" in the yellow tag, the title in blue and the yellow rule, as on the cover; on the right, the content right after the `=` (typically an `#image`), at the page margin from the edge, in a frame of 12em × 66% of the height of the content area: an image without a size fills it, an image with a size keeps it and shrinks when it is larger than the frame. The theme calls it for every `=`; it is not meant to be called by hand.

Argument

`config: (:)` — `dictionary`

Slide configuration, see `slide.config`.

Argument

`level: 1` — `int`

Heading level, passed by Touying.

Argument

`numbered: true` — `bool`

Passed by Touying; the part is always numbered.

Argument

`margin: auto` — `auto` | `length`

Distance between the content (what follows `=`, on the right) and the right edge of the page: `auto` is the page margin (`1.8em`, the same as the text), `0pt` puts it flush with the edge.

Argument

body `content`

Content that follows the `=` heading.

ending-slide()

```
ending-slide(
  title: auto auto none content
  subtitle: auto auto none content
  ..args content
) -> content
```

Closing slide: the blue page of the cover, with a large title with the yellow rule under it, a subtitle in yellow and a free body, centered; and the footer of the slides in white (the logos, the title, the authors and the slide number). Without a title it says “Thank you!” in the language of the deck; `title: none` removes the title and the rule.

Two forms: up to three contents in order, `[title][subtitle][body]`, or the named `title:` and `subtitle:` with the content as the body.

```
#ending-slide()
#ending-slide[Good work!][Next lecture: functions][Read chapter 3.]
#ending-slide(title: [Questions?], subtitle: [Lab 4 on Friday])[*See you!*]
#ending-slide(subtitle: [Questions?])[#info("contact")]
```

Argument

`title: auto` `auto` | `none` | `content`

The title; `auto` is “Thank you!” in the language of the deck, `none` removes it.

Argument

`subtitle: auto` `auto` | `none` | `content`

The subtitle, in yellow; `auto` and `none` give none.

Argument

`..args` `content`

Up to three contents, `[title][subtitle][body]`, or the body alone with `title:` or `subtitle:`.

exercise-slide()

```
exercise-slide(
  title: [] content
  color: auto auto color
  icon: auto auto none str content
  body content
) -> content
```

Exercise slide: the red box of an exercise filling the slide, with a pencil and “Exercise 1.N — Title” on its badge, numbered automatically. The function form of `== Title <exercise>`. Only the heading form can carry a unique label (`== Title <exercise-tides>`) and be cited with `@exercise-tides`, which shows “Exercise 1.N” linked to the slide.

```
#exercise-slide(title: [Tide table])[Find the next high tide.]
#exercise-slide(title: [Lab],
```

```
  color: colors.primary,
  icon: "terminal",
)[Another color and icon.]
```

Argument

`title: []` `content`

Text after the number, following a dash; empty omits the dash.

Argument

`color: auto` `auto` | `color`

Color of the box; `auto` is red (`tertiary`).

Argument

`icon: auto` `auto` | `none` | `str` | `content`

Icon before the name: the name of an `icon`, any content (an image) or `none` for no icon; `auto` is the icon of the theme, `ufac-theme(exercise-icon:)` (a pencil).

Argument

`body` `content`

Content of the slide; `---` or `#pagebreak()` inside it continues the same exercise.

example-slide()

```
example-slide(
  title: [] content
  color: auto auto color
  icon: auto auto none str content
  body content
) -> content
```

Example slide: the green box of an example, with a beaker and "Example 1.N — Title" on its badge, with a counter independent from the exercises. The function form of `== Title <example>`; see `exercise-slide()`.

```
#example-slide(title: [Wave speed])[Longer waves travel faster in deep water.]
```

Argument

`title: []` `content`

Text after the number, following a dash; empty omits the dash.

Argument

`color: auto` `auto` | `color`

Color of the box; `auto` is green (`quaternary`).

Argument

`icon: auto` `auto` | `none` | `str` | `content`

Icon before the name: the name of an `icon`, any content (an image) or `none` for no icon; `auto` is the icon of the theme, `ufac-theme(example-icon:)` (a beaker).

Argument

body `content`

Content of the slide.

`solution()`

```
solution(
  sep: "." str content
) -> content
```

The marker of the start of the solution of an example, alone on its line like Touying's `#pause`: what comes before it is the statement and what comes after it the solution, opened by a dashed line and "Solução." / "Solution." / "Solución." in bold. With a `#pause` right before or right after it, the solution appears on the next subslide, opening included. Written alone, `#solution`, the word ends with a period; `#solution(sep: ":")` ends it with another separator.

```
== The state of water <example>
A script that reads the temperature and prints the state of the water.
#pause
#solution
Every condition is an `if`: …
```

Argument

sep: "." `str` | `content`

What follows the word, in bold: `"."`, `":"`, `[:\ ]` (a colon and a line break) or any content.

`answer()`

```
answer(
  sep: "." str content
) -> content
```

The marker of the start of the answer of an exercise: the same as `solution()`, opened by "Resposta." / "Answer." / "Respuesta.", with the same separator.

Argument

sep: "." `str` | `content`

What follows the word, in bold: `"."`, `":"`, `[:\ ]` or any content.

12.2 The information of the deck

`info()`

```
info(
  key str function
  height: auto auto length
  white: false bool
) -> content
```

A field of the deck, ready to show, anywhere in a slide (inside boxes and tables too): `#info("author")` gives the authors as written, or "A, B and C" for an array; `#info("date")` the date in full; `#info("logo", height: 2em)` the logos. The aliases are keys too: `#info("subject")` is the `title`. With a

function, it receives the dictionary of all the information, with the keys above plus `authors` and `institutions` (arrays): `#info(i => [#i.short-title: #i.title])`.

```
The course #info("title"), taught by #info("author").
```

Argument

key — `str` | `function`

`"title"`, `"short-title"`, `"subtitle"`, `"short-subtitle"`, `"author"`, `"date"`, `"institution"`, `"logo"`, `"contact"` or an alias (`"subject"`, `"semester"`, …); or a function of the dictionary.

Argument

`height: auto` — `auto` | `length`

Only for `"logo"`: the height of the logos; `auto` is the height in the footer (0.64em).

Argument

`white: false` — `bool`

Only for `"logo"`: white logos, as on the cover.

12.3 Boxes and emphases

emph-box()

```
emph-box(
  color: colors.primary color
  title: none none content
  icon: none none str content
  body content
) -> content
```

Emphasis box. Without a title it has a thin border; with a title, a badge sits over the top border. Inside the box the `-` markers, the `->` arrow, `*bold*`, `==highlight==` and `#underline` take the box color; an explicit `#arrows(color:)`, `#highlight(fill:)` or `#underline(stroke:)` wins. Colors by convention: primary for definitions and what is recalled, quaternary for advantages, tertiary for problems, secondary for side notes.

```
#emph-box(color: colors.quaternary,
  title: [Pros])[
  - Fast and *simple*
  - Works with ==highlights==
]
#emph-box(color: colors.tertiary,
  icon: "exercise", title: [Homework])[
  An icon before the title.
]
```

Pros

- Fast and **simple**
- Works with highlights

Homework

An icon before the title.

Shorthand, always in the primary color: a paragraph whose lines start with `>`; a first line `> == Title` gives the title. Only inline content fits the shorthand. For a framed equation see `eq-box()`.

Argument

`color: colors.primary` — `color`

Color of the border and badge, and of the bold text, markers, arrows and highlights inside.

Argument

`title: none` `none` | `content`

Badge over the top border; `none` gives the thin-border box (unless there is an `icon`).

Argument

`icon: none` `none` | `str` | `content`

Icon at the start of the badge, before the title: the name of an `icon()`, any content (an image, a symbol) or `none`. It is centered on the capitals of the title, as in the badges of exercises and examples; without a title it is the whole badge. The badge has the same height with or without an icon.

Argument

`body` `content`

Content of the box.

eq-box()

```
eq-box(
  color: auto  auto  color
  body  content
) -> content
```

Equation box: a light frame around a key equation. It goes **inside** the equation, like the `boxeq` of the bookly package, so the equation keeps its centering, spacing and numbering, and a part of an equation can be framed too. The body is written as math, between dollar signs; it is set in display style whatever the spacing around it. A box around the whole equation is padded generously, a box around a part of one tightly, and in both cases the frame holds tall fractions and the limits of sums.

It also works in a line of text or in an inline equation: there the body is in text style and the box does not change the line spacing.

```
The area #eq-box[$A = pi r^2$] and the
mean #eq-box[$1/N sum_i x_i$].
```

The area $\boxed{A = \pi r^2}$ and the mean $\boxed{\frac{1}{N}\sum_i x_i}$.

```
$ #eq-box[$E = m c^2$] $

$ F = #eq-box(
  color: colors.quaternary,
)[$m a$] + f $
```

$$\boxed{E = mc^2}$$

$$F = \boxed{ma} + f$$

Shorthand, always in the primary color: `>` in front of a block equation, on the same line or alone on the line before it.

```
> $ sigma^2 = 1/n sum_i (x_i - mu)^2 $
```

$$\boxed{\sigma^2 = \frac{1}{n}\sum_i (x_i - \mu)^2}$$

Argument

`color: auto` `auto` | `color`

Base color of the fill and border; `auto` is blue, or the box color inside an `emph-box()`.

Argument

body `content`

The equation, as math: `[$x = 2$]` or `($x = 2$)`.

callout()

```
callout(
  kind str
  style: "bar" str
  title: auto auto none content
  color: auto auto color
  icon: auto auto none str content
  body content
) -> content
```

Callout: a box with the color, the icon and the title of its kind, the title in the language of the deck. The default style, `"bar"`, is the box of a printed handout: a bar on the left, a light fill, the icon and the title in bold opening the text ("**Tip.** …", "**Caution!** …"); `"badge"` is an `emph-box()` with the title on a badge over the border. Inside it, as in a box, the bold text, the markers and the arrows take its color; a code block keeps its own look. Each kind has a function of its own, with the same options: `caution()`, `tip()`, `remark()` and `note()`.

```
#callout("caution")[
  `x*y` with two vectors is an error.
]
#callout("tip", style: "badge")[
  The style with the badge.
]
#callout("note", title: [Aside.])[
  Another title.
]
```

Caution! `x*y` with two vectors is an error.

Tip

The style with the badge.

Aside. Another title.

Argument

kind `str`

`"caution"` (red, a mistake to avoid), `"tip"` (green, a shortcut), `"remark"` (blue, to recall what was seen: "Relembrando" / "Reminder" / "Recordando") or `"note"` (gray, something secondary).

Argument

style: "bar" `str`

`"bar"` (a bar on the left and the title opening the text) or `"badge"` (the title on a badge over the border).

Argument

title: auto `auto` | `none` | `content`

The title; `auto` is the name of the kind in the language of the deck ("Caution" / "Atenção"; in the style `"bar"` with its punctuation, "Caution!", "¡Precaución!", "Tip."); `none` leaves the icon alone.

Argument

`color: auto` — `auto` | `color`

The color; `auto` is the one of the kind.

Argument

`icon: auto` — `auto` | `none` | `str` | `content`

The icon: a name of `icon`, content or `none`; `auto` is the one of the kind.

Argument

`body` — `content`

The text.

caution()

```
caution(
  ..args arguments
) -> content
```

A caution, red, for a mistake to avoid: `#caution[..]` is `#callout("caution")[..]`, with the same options (`style`, `title`, `color`, `icon`).

```
#caution[
  `x*y` with two vectors is an error.
]
```

> **Caution!** `x*y` with two vectors is an error.

Argument

`..args` — `arguments`

The options of `callout()` and the text.

tip()

```
tip(
  ..args arguments
) -> content
```

A tip, green: `#tip[..]` is `#callout("tip")[..]`, with the same options.

```
#tip[The up arrow recalls a command.]
```

> **Tip.** The up arrow recalls a command.

Argument

`..args` — `arguments`

The options of `callout()` and the text.

remark()

```
remark(
  ..args arguments
) -> content
```

A reminder of what was seen before ("Relembrando" / "Reminder" / "Recordando"), blue: `#remark[..]` is `#callout("remark")[..]`, with the same options.

```
#remark[
  A *function* is a named block of code.
]
```

Reminder. A **function** is a named block of code.

Argument

`..args` `arguments`

The options of `callout()` and the text.

note()

```
note(
  ..args arguments
) -> content
```

A note, gray, for what is secondary: `#note[..]` is `#callout("note")[..]`, with the same options.

```
#note[Two versions of the program
  differ here.]
```

Note. Two versions of the program differ here.

Argument

`..args` `arguments`

The options of `callout()` and the text.

tag()

```
tag(
  color: colors.secondary color
  size: 0.6em length
  body content
) -> content
```

Tag: capitals, spaced and bold, on a color, with rounded corners; by default UFAC yellow with the text in dark blue, as on the cover ("EXTENSION COURSE") and on the section slides ("PART 2"). The text is dark blue or white, whichever contrasts more with the color. The vertical padding is an `outset`, as in `eq-box()`: in a line of text the tag does not change the height of the line.

```
-> `plot3` #tag[new]
-> #tag(color: colors.tertiary)[exam]
-> #tag(
  color: colors.quaternary-light,
)[optional]
```

→ plot3 NEW
→ EXAM
→ OPTIONAL

Argument

`color: colors.secondary` `color`

The color of the tag.

Argument

`size: 0.6em` `length`

The size of the text, relative to the text around it.

Argument

body `content`

The text, in capitals.

quote-box()

```
quote-box(
  color: colors.secondary color
  body content
) -> content
```

Quotation box: a vertical bar on the left. Typst's `#quote(attribution: [Author])[…]` is shown as a `quote-box` too, with the attribution on the right.

```
#quote-box(color: colors.primary)[
  All models are wrong, but some
  are useful.
]
```

> All models are wrong, but some are useful.

Shorthand: a paragraph made only of `"quoted"` runs, one per line; each run becomes a box and the colors cycle secondary, primary, quaternary, tertiary. Quotes inside a sentence are left alone.

Argument

`color: colors.secondary` `color`

Color of the bar.

Argument

body `content`

The quotation.

primary()

```
primary(
  body content
) -> content
```

Bold text in UFAC blue: the same as `#alert` (Touying's method) and as the `~~text~~` shorthand. Inside a pill (a `===` subtitle, the badge of an exercise, the title of an `emph-box()`) it turns UFAC yellow, where blue would vanish.

```
A #primary[definition], the same as
~~this shorthand~~.
```

A **definition**, the same as **this shorthand**.

Argument

body `content`

Text to emphasize.

`secondary()`

```
secondary(
  body content
) -> content
```

Bold text in UFAC yellow. It is weak on white: meant for short labels and dark backgrounds.

```
A #secondary[side note] in yellow.
```

A **side note** in yellow.

Argument

body `content`

Text to emphasize.

`tertiary()`

```
tertiary(
  body content
) -> content
```

Bold text in red, for warnings and problems.

```
#tertiary[Caution:] the ice is thin.
```

Caution: the ice is thin.

Argument

body `content`

Text to emphasize.

`quaternary()`

```
quaternary(
  body content
) -> content
```

Bold text in green, for advantages and correct answers.

```
#quaternary[Correct:] the series
converges.
```

Correct: the series converges.

Argument

body `content`

Text to emphasize.

12.4 Lists, columns and the outline

`arrows()`

```
arrows(
  color: colors.primary color
  body content
) -> content
```

Arrow lists with the arrow in another color, for everything inside the body. The `-> item` shorthand itself needs no function: consecutive lines starting with `->` become one list with a blue arrow (the box color inside an `emph-box()`). The list must start the paragraph, and `-\> item` escapes it.

```
#arrows(color: colors.quaternary)[
  -> green here
  -> and here
]
```

→ green here
→ and here

Argument

`color: colors.primary` `color`

Color of the arrows.

Argument

body `content`

Content with `-> item` paragraphs.

cols()

```
cols(
  columns: auto auto array
  gutter: 1em length
  divider: false bool
  ..bodies content
) -> content
```

Columns: a thin wrapper over Touying's `cols` (a `grid`), with an optional divider. Touying's own `side-by-side` (no divider) is still available.

```
#cols(divider: true,
  columns: (1fr, 2fr))[
  Left
][
  -> the shorthands work
  -> inside a column
]
```

Left

→ the shorthands work
→ inside a column

Argument

`columns: auto` `auto` | `array`

Column widths, as Touying accepts them: `auto` (equal widths) or an array such as `(1fr, 2fr)`.

Argument

`gutter: 1em` `length`

Space between the columns.

Argument

`divider: false` `bool`

A blue vertical bar (2.2pt) between the columns; the gutter is then used as inset.

Argument

`..bodies` `content`

One content block per column.

`outline()`

```
outline(
) -> content
```

Outline of the deck: its parts (the `= Title` headings, the "Part N" of the section slides), one per line, with the number in a small pill, the title in bold blue and, after a dotted leader, the number of the section slide (the slide number of the footer, which skips the cover); every line is a link to that slide. In a deck it takes the place of Typst's `outline`, which is still `std.outline`. It goes on a slide of its own, usually the first after the cover:

```
== Contents
#outline()
```

12.5 Code

`code-languages` `dictionary`

The languages the theme knows: the name in the pill of a block and its icon (a name of `icon`, content or `none`). The languages with a logo show it; the shell (`bash`, `sh`) shows a colored terminal. `ufac-theme(code-languages:)` adds languages or replaces any of them except the output, in the same format. The output (`text`, `txt`, `output`) is "Saída" / "Output" / "Salida" in the language of the deck (`name: auto`).

`ufac-syntax` `bytes`

The syntax colors of the code blocks: a tmTheme with the palette of the theme and general scopes (comments in green, strings in red, keywords in blue, numbers in ochre), which suits any language.

`codly-ufac-theme` `dictionary`

The UFAC theme for codly 2, a dictionary in the format of its themes, for `codly.theme(..)`: the theme exports only data and codly applies it. The blue box (the bar on the left is drawn by a rule of the theme, since codly's border is the same on the four sides), the line numbers in light blue, the file name on a light blue band, the highlights in the colors of the theme and the syntax colors of `ufac-syntax`. `ufac-theme(codly: codly)` applies it; it is exported for a document that applies codly by itself.

12.6 Icons

`icon()`

```
icon(
  name str
  color: auto auto color
  size: 1em length
  baseline: 25% relative
) -> content
```

An icon, already wrapped in a `box` for inline use: a role of the theme (`"caution"` or `"warning"`, `"tip"`, `"info"`, `"note"`, `"exercise"`, `"example"`, `"file"`, `"terminal"`, `"contact"`, `"output"`, `"code"`, `"python"`, …), one of the Octicons the theme ships by its own name (`"alert"`, `"light-bulb"`, `"pencil"`, `"beaker"`, …) or any icon of a collection given with `provide-icons`, as `"collection:name"` (`"octicon:rocket-16"`, `"mdi:home"`). An unknown name is a compile error that says how to get the icon.

```
#icon("tip", color: colors.quaternary)
A tip,
#icon("warning") attention and a big
#icon("python", size: 1.6em)
```

A tip, attention and a big

Argument

`name` `str`

Name of the icon.

Argument

`color: auto` `auto` | `color`

Fill color (multicolor logos keep their colors). With `auto` it follows the context like bold text: white inside a pill (a `===` subtitle, the badge of an exercise, the title of an `emph-box()`), the box or title color inside `emph-box()` and `==`, black elsewhere.

Argument

`size: 1em` `length`

Height of the icon; the width follows its proportions (square for the icons of the theme).

Argument

`baseline: 25%` `relative`

Baseline of the box; the default aligns the icon with the text.

`provide-icons()`

```
provide-icons(
  ..collections dictionary
) -> content
```

Gives the deck more icons: one or more collections in the JSON format of Iconify (`json("mdi.json")`), whose icons are then written `#icon("mdi:home")`. It is the `provide-icons` of the iconify package.

```
#provide-icons(json("octicon.json"))
#icon("octicon:rocket-16")
```

Argument

`..collections` `dictionary`

The collections, each a dictionary read from a JSON file of Iconify.

`theme-icons` `dictionary`

The icon of each role of the theme, as `"collection:name"`: Octicons (size 16) for the callouts, the badges of exercises and examples, the file name of a code block, the terminal and the contact; Phosphor in the bold weight for code; colored icons for the pills of the code blocks: the logos of Devicon (Typst from Simple Icons), the terminal of Material Icon Theme for the shell and the log of VSCode Icons for the output of a program. A role is also a valid name for `icon`: `#icon("caution")`. `warning` and `caution` are the same icon, under the name of 0.2.0 and the one of the callout.

12.7 Colors and logo

`colors` `dictionary`

The theme palette: the official UFAC colors plus a neutral gray, named after Touying's `config-colors` keys (`primary`, `secondary`, `tertiary`, `neutral`) plus `quaternary`. Each family has 7 tones,

`<family>-darkest`, `-darker`, `-dark`, the base, `-light`, `-lighter` and `-lightest`, so `colors.primary-lighter`, `colors.quaternary-dark` etc. exist. The whole dictionary is passed to `config-colors`, hence the same keys are available as `self.colors.*`.

Family	Base	Role
`primary`	`#0C4DA2`	titles, `->` markers, tables, `#alert`/`#primary`, pills
`secondary`	`#FFBF14`	`-` markers, `#secondary`, quotes; `-lighter` is the highlight
`tertiary`	`#BE1E2D`	`#tertiary`, exercises
`quaternary`	`#09B081`	`#quaternary`, examples
`neutral`	`#808080`	`-darkest` is the bold/italic text, `-lightest` the background

`ufac-logo` `str`

Official UFAC logo as an SVG string (yellow shield and blue lettering). In a deck, `#info("logo")` shows it in the colors of the theme (`height:`, and `white: true` on a dark background); outside one, as in a handout that goes with the slides, `image(bytes(ufac-logo))` draws it.

The logo belongs to the Universidade Federal do Acre and is not covered by the license of the package.

```
#image(bytes(ufac-logo), height: 2em)
```



13 Index of definitions

answer() p. 38
arrows() p. 45
callout() p. 41
caution() p. 42
code-languages p. 47
codly-ufac-theme p. 47
colors p. 48
cols() p. 46
emph-box() p. 39
ending-slide() p. 36
eq-box() p. 40
example-slide() p. 37
exercise-slide() p. 36
icon() p. 47
info() p. 38
new-section-slide() p. 35
note() p. 43
outline() p. 47
primary() p. 44
provide-icons() p. 48
quaternary() p. 45
quote-box() p. 44
remark() p. 42
secondary() p. 45
slide() p. 34
solution() p. 38
tag() p. 43
tertiary() p. 45
theme-icons p. 48
tip() p. 42
title-slide() p. 33
ufac-logo p. 49
ufac-syntax p. 47
ufac-theme() p. 31